

Signature Verification system

PROJECT REPORT

Submitted in partial fulfillment of the requirements
for the degree of Bachelor of Technology in

Electronics and Communication Engineering

Triguna Sen School of Technology, Assam University, Silchar



Submitted by:

Tasnim Shamim

Roll No: 1180100253

Batch: 2022–2026

Under the Guidance of

Dr. Richik Kashyap

Department of Electronics and Communication Engineering
Assam University, Silchar

Declaration

I hereby declare that the project report entitled “**Signature Verification System**” submitted to the Department of Electronics and Communication Engineering, Assam University, Silchar, in partial fulfillment of the requirements for the degree of Bachelor of Technology, is a record of original work carried out by me under the supervision of **Dr. Richik Kashyap**.

The results presented in this report are based on my own work and have not been previously submitted for any other degree or diploma to this or any other institution.

Tasnim Shamim

Roll No: 1180100253

B.Tech, ECE (2022–2026)

Assam University, Silchar

Signature

Certificate

This is to certify that the project entitled **Signature Verification System**, submitted by **Tasnim Shamim** to Assam Univeristy,Silchar for the award of the degree of **Bachelor of Technology (ECE)** is a bonafide record of the project work carried out by them under my supervision and guidance. The content of the project, in full or parts have not been submitted to any other institute or university for the award of any degree or diploma.

Dr. Richik Kashyap

Assitant Professor

Department of ECE

Assam University, Silchar

Signature & Seal

Contents

Declaration	i
Certificate	ii
List of Figures	v
List of Tables	vi
Acknowledgement	vii
Abstract	viii
1 Introduction	1
1.1 Introduction	1
2 Literature Review	3
3 Methodology	11
3.1 Data Preprocessing	11
3.2 Model Architecture	12
3.3 Training Procedure	13
4 Result and Discussion	16
4.1 Environmental Setup	16
4.2 Dataset Description	16
4.3 Evaluation Metrics	17
4.3.1 Euclidean Distance	17
4.3.2 Contrastive Loss	17
4.3.3 Accuracy, Precision, Recall and F1-Score	18
4.3.4 ROC Curve and AUC	18

4.4	Experiments and Results	19
4.4.1	Experimental Setup	19
4.4.2	Threshold Selection	20
4.4.3	Performance Evaluation	20
4.4.4	Training Analysis	20
4.4.5	Confusion Matrix	22
4.4.6	ROC Curve	22
4.4.7	Prediction Results	23
4.4.8	Discussion	24
5	Web Application	25
6	Conclusion	28
7	Future Work	x

List of Figures

3.1	Similar V/S Disimilar Count Plot	12
3.2	EfficientNetB0 Architecture	13
3.3	Proposed Siamese Network Architecture for Signature Verification. Two input images are processed through identical EfficientNetB0 networks with shared weights. The extracted feature embeddings are compared using Euclidean distance, followed by a classification layer to determine similarity. . . .	14
4.1	Training and validation accuracy and loss curves	21
4.2	Confusion Matrix of the Model	22
4.3	ROC Curve showing model performance	22
4.4	Sample Predictions (Genuine vs Forged)	23
5.1	Web Application Architecture for Signature Verification System	25
5.2	Web interface showing signature upload page	26
5.3	Web interface showing signature upload page	27
5.4	Web interface showing signature upload page	27
5.5	Web interface showing signature upload page	27

List of Tables

2.1	Summary of Related Works	10
4.1	Hyperparameters Used During Training	19
4.2	Performance metrics of the proposed model on the test dataset	20

Acknowledgement

I would like to express our heartfelt gratitude to our esteemed guide, **Dr. Richik Kashyap**, Assistant Professor at Assam University, Silchar for his invaluable guidance, unwavering support, and encouragement throughout our project journey. His profound knowledge, insightful suggestions, and mentorship have been instrumental in shaping our project and academic pursuits.

I would like to express our heartfelt gratitude to our HOD, the esteemed Vice-Chancellor, the class teachers, and all the faculty members of our department for their unwavering support and guidance throughout our project journey. Your invaluable contributions have been instrumental in shaping this project endeavor.

I am deeply indebted to our family members whose constant love, encouragement, and support have been the driving force behind our academic pursuits. Their unwavering belief in us has given us the strength to overcome challenges and achieve milestones.

I also want to express our heartfelt gratitude to our friends, colleagues, juniors, and seniors for their camaraderie, encouragement, and support throughout this academic journey. Their presence and interactions have enriched us with diverse perspectives, making this journey all the more rewarding.

Tasnim Shamim

Abstract

This project presents an automated signature verification system using a deep learning-based Siamese Network architecture. The system aims to address the limitations of traditional manual signature verification methods, which are often time-consuming and prone to human error. Leveraging the EfficientNet-B0 backbone for feature extraction, the proposed model learns discriminative representations of signatures and determines similarity between pairs using Euclidean distance. The system is trained on the CEDAR signature dataset, consisting of both genuine and forged samples. A contrastive loss function is employed to effectively separate similar and dissimilar signature pairs in the embedding space. Various preprocessing techniques, including resizing, normalization, and augmentation, are applied to improve robustness and generalization. Experimental results demonstrate strong performance, achieving an accuracy of 97.24% along with low False Acceptance Rate (FAR) and False Rejection Rate (FRR). Additionally, a Streamlit-based web application is developed to enable real-time signature verification, making the system practical for real-world deployment. Overall, the proposed approach provides a scalable, efficient, and reliable solution for signature verification tasks, with potential applications in banking, legal authentication, and forensic analysis.

Keywords: Handwriting Recognition, Siamese Network, EfficientNet-B0, Contrastive Loss, Writer Verification, Deep Learning, Biometrics, OCR.

Chapter 1

Introduction

1.1 Introduction

The signature verification system developed in this project addresses the growing need for reliable and automated identity validation across multiple domains. In sectors such as banking, legal services, and fraud detection, verifying the authenticity of signatures is critical for ensuring security and trust. Traditional manual verification methods are time-consuming and prone to human error, making automated solutions increasingly important. Beyond security applications, such systems can also assist in forensic investigations and document validation tasks where signature authenticity plays a crucial role.

Handwriting, particularly signatures, is a distinctive behavioral biometric that varies across individuals while maintaining certain consistent patterns for each person. However, natural variations in a person's signature due to factors such as writing speed, pressure, and environment make the verification task challenging. This project focuses specifically on offline signature verification, where signatures are obtained from scanned or photographed documents rather than real-time capture devices.

With advancements in deep learning, Convolutional Neural Networks (CNNs) have shown strong performance in image-based recognition tasks. In this work, a Siamese Network architecture with an EfficientNet-B0 backbone is employed to learn a similarity function between pairs of signature images. Instead of classifying signatures into fixed categories, the model determines whether two signatures belong to the same individual by learning discriminative feature representations. This approach enables better generalization to unseen signatures compared to traditional classification-based methods. The model is trained using contrastive learning on paired signature data, allowing it to effectively distinguish between genuine and forged signatures. Performance is evaluated using verification-specific metrics such as accuracy, False Acceptance Rate (FAR), and False Rejection Rate (FRR). The overall

system is designed to provide a scalable and efficient solution for real-world signature verification tasks while maintaining robustness against variations and potential forgeries. The rest of this report is structured as follows. Chapter 2 presents a review of related literature on handwriting recognition and signature verification Chapter 3 describes the methodology used. Chapter 4 gives the Result and Discuss Section. Chapter 5 provides Web Application Details Chapter 6 Concludes the Project Report and Chapter 7 gives us idea about the future work directions.

Chapter 2

Literature Review

Handwriting recognition has been an active area of research in artificial intelligence and pattern recognition for over three decades. The field has evolved from early rule-based and template-matching systems to sophisticated deep learning architectures capable of achieving near-human accuracy on standard benchmarks. This chapter reviews the major developments in handwriting recognition, focusing on methods relevant to writer identification and verification. Early handwriting recognition systems relied entirely on handcrafted features. These included stroke width, slant angle, letter spacing, pen pressure, and baseline alignment. Techniques such as Hidden Markov Models (HMMs), Dynamic Time Warping (DTW), and Support Vector Machines (SVMs) were commonly used for classification [10].

While these methods established a foundation, they were limited by the difficulty of manually engineering features that could generalize well across writers, scripts, and styles. They also struggled with cursive writing and natural variations in a single writer's style over time.

The application of machine learning brought improvements in scalability and accuracy. SVM-based recognition systems were widely adopted due to their strong theoretical foundation and performance on structured data. Hamdan and Sathesh [10] developed a statistical SVM-based model for handwritten character recognition, achieving 88% accuracy across diverse datasets including MNIST, IAM, and UCOM.

Chychkarov et al. [6] compared SVM, k-Nearest Neighbors (kNN), Random Forest (RF), and deep learning neural networks for digit recognition using MNIST, with their best model achieving 97.6% accuracy.

The introduction of deep learning, particularly Convolutional Neural Networks (CNNs), marked a turning point in handwriting recognition. CNNs automatically learn hierarchical feature representations from raw image data, eliminating the need for manual feature engineering.

Siddique et al. [18] demonstrated that CNNs with optimized hidden layer configurations achieved 99.21% accuracy on MNIST handwritten digit recognition. Saqib et al. [17] proposed a purpose-built CNN architecture for handwritten character recognition that achieved remarkable accuracies of 99.642% for digits and 99.563% for alphabets.

For Arabic scripts, Ahamed et al. [1] employed CNN with morphological data augmentation, expanding their dataset from 3,000 to 72,000 images and achieving 98.91% accuracy. Ali and Mallaiah [3] combined CNN with SVM classifiers and dropout regularization for Arabic handwriting, achieving 99.85% accuracy on the HACDB dataset.

For sequence-based handwriting recognition at the word and line level, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks have proven effective. Noubigh et al. [15] integrated CNN with Bidirectional LSTM (BLSTM) for Arabic handwriting recognition on the KHATT database, achieving 8% Character Error Rate (CER) and 20.1% Word Error Rate (WER).

Chandio et al. [5] proposed a deep Convolutional Recurrent Neural Network (CRNN) for cursive Urdu text recognition, achieving 95.75% character recognition rate. Document-level recognition was addressed by Coquenot et al. [7], whose Document Attention Network (DAN) combined a Fully Convolutional Network (FCN) encoder with a transformer decoder, achieving 3.43% CER on the READ 2016 dataset.

Siamese Networks were introduced as a solution to the *one-shot learning* problem, where a model must recognize new classes from very few examples. Koch et al. (2015) demonstrated their effectiveness for signature verification. Unlike classification CNNs that assign samples to fixed categories, Siamese Networks learn a distance metric to determine whether two samples belong to the same class.

This makes them particularly well-suited for writer verification, where it is impractical to include every possible writer as a training class. The network is trained on pairs of samples with a contrastive loss function that minimizes the distance between similar pairs and maximizes it for dissimilar ones.

Transfer learning using pretrained architectures such as VGG, ResNet, and EfficientNet has significantly improved performance, especially on smaller datasets. Parikshith et al. [16] used CNN with transfer learning for Kannada handwritten character recognition, achieving 96.7% accuracy on the

Chars74K dataset. EfficientNet, introduced by Tan and Le (2019), achieves state-of-the-art accuracy with substantially fewer parameters through compound scaling of network depth, width, and resolution, making it an ideal backbone for a Siamese architecture.

An important early contribution to full-page handwriting recognition was made by Kim, Govindaraju, and Srihari [12], who proposed a complete end-to-end system for reading handwritten page images. Their architecture comprised five tightly coupled functional modules: image conversion, line separation, preprocessing, word segmentation, word recognition, and linguistic post-processing.

The system used chain code representation to reduce the storage and processing demands of high-resolution page images, converting a standard 300 dpi US letter page from over 8 MB into a compact, manipulable form. Line separation was achieved by clustering the local minima of contour components to identify imaginary baselines — a technique that proved effective even when ascenders and descenders of adjacent lines overlapped. Preprocessing involved slant correction using vertical line elements extracted through one-dimensional filters, followed by chain code smoothing to eliminate digitization noise.

A key innovation was the intelligent word segmentation module, which trained a neural network on eight geometric features derived from character segment intervals and heights. Unlike earlier approaches that relied solely on physical gap distances between components, this method modeled writer-specific spacing patterns, resulting in significantly higher segmentation accuracy. The word recognition module employed a lexicon-driven matching scheme using variable-duration alignment, which balanced speed and accuracy for lexicons of approximately 1,600 to 50,000 words. Linguistic post-processing used sentence-level context to override erroneous top word choices, demonstrating that contextual constraints are essential when lexicon size is large.

Evaluated on twenty handwritten page images at 300 dpi, the system achieved line separation accuracy of 95.3%, word segmentation accuracy of 94.5%, and top-10 word recognition accuracy of 79.8%. The timing profile revealed that word recognition and linguistic post-processing together consumed over 70% of total processing time, highlighting these as the primary bottlenecks for future optimization. This work remains a foundational reference for pipeline-based handwriting recognition and directly motivates the preprocessing and segmentation strategies adopted in subsequent systems, including the present

project.

A distinct but closely related research direction concerns the automatic classification of handwriting style as a preprocessing step before recognition. Dehkordi, Sherkat, and Allen [8] proposed a style classifier designed to route handwriting samples to the most appropriate recognizer based on a machine-specific definition of legibility, rather than a human perceptual definition.

The approach operates at the word level and extracts a rich set of 36 contour-based features from handwritten word images. These features are organized into four categories. Global features capture overall shape statistics such as the ratio of sharp angles, averaged component lengths, and the proportions of vertical, horizontal, and diagonal stroke directions derived from Freeman chain codes. Region-based features measure the distribution of convex, concave, and plain contour segments by partitioning each word contour after Gaussian smoothing and identifying dominant corner points. Window-based features divide the word image into three horizontal zones — upper, middle, and lower — and compute directional slope ratios within each zone, capturing how ink density and orientation vary vertically across the word. Finally, a zero-crossing feature counts the intersections of a horizontal midline with the word contour, and a first-moment feature captures global shape asymmetry.

Two classification techniques were evaluated. Multiple Discriminant Analysis (MDA) transforms the 36-dimensional feature space into an optimal lower-dimensional discriminant space and classifies using a nearest-mean rule. The Probabilistic Neural Network (PNN) method, based on Bayesian decision theory with Parzen window density estimation, operates non-parametrically and does not assume a particular distribution for the feature data. The PNN was optimized by selecting the kernel bandwidth parameter σ through leave-one-out cross-validation.

Experiments on 2,974 word images from 18 writers showed that PNN consistently outperformed MDA across all classification tasks. For binary classification on unseen test data, PNN achieved 86.5% accuracy distinguishing legible from illegible handwriting, 90.5% for middle versus illegible, and 65.5% for legible versus middle. For the more challenging three-class problem (legible, middle, illegible), PNN achieved 67.33% accuracy on unseen data. MDA, by contrast, performed only marginally above chance on most binary tasks (around 60–65%), demonstrating its sensitivity to non-normal and multimodal feature distributions.

The authors observed that the “middle” class was the hardest to separate from both the legible and illegible classes, suggesting it represents an inherently ambiguous boundary that resists crisp geometric characterization. They also noted that all 36 features contributed to classification performance, with significant reduction in feature count leading to substantially worse results. This work is directly relevant to the present project, as automatic style characterization can inform preprocessing decisions and select the most appropriate verification threshold for a given writer’s style.

A major practical challenge in deploying HTR systems at scale is the scarcity of labeled offline handwriting data, particularly for less-resourced languages and scripts. Ingle et al. [11] from Google Research addressed this challenge by proposing a framework that leverages large volumes of *online* handwriting data — stroke trajectories captured as (x, y) coordinate sequences from mobile devices — to train offline handwritten text recognition models, eliminating the need for expensive manual annotation of handwriting images.

The core contribution is a data generation pipeline that converts online ink data into realistic offline training images. Raw stroke data is first processed through forced alignment for character segmentation, then rendered into binary images with randomized stroke width, slant, and boundary size. Short individual word inks are concatenated into longer synthetic text lines to provide more realistic training sequences. A multi-step image degradation pipeline then applies transformations including background superimposition, contrast variation, Gaussian blur, additive noise, spline distortions, and JPEG compression artifacts, making the rendered images closely approximate the appearance of real scanned handwriting.

To compensate for the dominance of print-style writing in the mobile data, the authors additionally trained a handwriting synthesis model based on a mixture density recurrent network with a style embedding module. This module captures the writing style of a given sample and allows new cursive-style strokes to be synthesized in that style, enriching the diversity of the training data.

For the line recognition model, two architectures were evaluated. The first is an LSTM-based model using stacked bidirectional LSTMs (BLSTMs) following an inception-style CNN encoder. The second replaces the BLSTM layers with Gated Recurrent Convolutional Layers (GRCLs), which simulate recurrency along network depth rather than time, allowing fully feed-forward inference that is more amenable to parallel processing on GPUs and TPUs.

Experiments on the IAM Offline benchmark showed that models trained on rendered online data outperformed those trained on native offline data, owing to the larger effective dataset size and the regularizing effect of random degradations. Combining both sources reduced the Character Error Rate to 5.7% (BLSTM) and 6.7% (GRCL) on the IAM test set. The fully tuned GRCL system achieved 4.0% CER, approaching state-of-the-art performance.

Integration into a production OCR pipeline was achieved through a dual-head script and style classification model. A shared CNN encoder feeds two classification heads — one identifying the writing script and the other distinguishing printed from handwritten text. The handwriting-specific recognizer is then activated when the style head exceeds a confidence threshold. This paper is directly relevant to the present project as it demonstrates that leveraging online data and synthetic augmentation are viable strategies for reducing the data bottleneck in handwriting verification systems, and that feed-forward architectures like GRCL can match recurrent models in accuracy while being more deployment-friendly.

Balci, Saadati, and Shiferaw [4] investigated two complementary approaches to handwritten word recognition using the IAM Handwriting Dataset: direct word-level image classification and character-level segmentation followed by classification. Their work provides a clear empirical comparison of these strategies and highlights both the promise and limitations of purely CNN-based classification without sequence modeling.

For word-level classification, the authors trained VGG-19, ResNet-18, and ResNet-34 architectures on a vocabulary of 50 words, using cross-entropy loss with the Adam optimizer. Input images were standardized through a preprocessing pipeline that included whitespace padding to uniform dimensions, small random rotations to model real-world tilt, and mean pixel subtraction for zero-centering. The rationale for padding over cropping or resizing was carefully argued: cropping risks removing characters, while non-uniform rescaling distorts the aspect ratio and loses inherent shape information. ResNet-34 achieved the best word-level performance at 35% training accuracy and 25% test accuracy, while VGG-19 achieved only 20% test accuracy, reflecting the known superiority of residual connections for deep classification tasks.

The authors observed that word-level classification fundamentally suffers from the enormous vocabulary size of natural language — over 10,000 words

in their training set alone — which makes the softmax output space intractably large and fine-grained visual differentiation between word images extremely difficult. This motivated a shift to character-level classification. A fine-tuned Tesseract 4.0 LSTM-based segmentation model was used to extract bounding boxes for individual characters within word images, which were then independently classified using the same CNN architectures. Character-level classification achieved 31% test accuracy, outperforming all word-level models despite the added segmentation step.

Key observations from this work include: the critical importance of data volume for deep learning (the authors estimate 10^5 – 10^6 training examples are needed for strong performance), the difficulty of segmenting cursive characters where character boundaries are ambiguous, and the utility of beam search with a language model as a decoding strategy for character sequences. These findings reinforce the design choices in the present project — specifically, the adoption of a similarity-based pairwise Siamese approach rather than a fixed-class softmax classifier, and the recognition that sequence-level context modeling (via CTC or attention) is necessary for production-grade handwriting systems.

Table 2.1 summarizes the key related works reviewed in this chapter. The literature reflects a clear evolution from handcrafted feature methods and statistical classifiers, through CNN-based approaches, to modern end-to-end deep learning architectures. While character and word-level recognition has reached high accuracy on standard benchmarks, writer verification using similarity learning — particularly with modern efficient backbones — remains an area with significant room for development. Furthermore, combining style classification as a preprocessing step with verification models is a largely unexplored direction. The present project addresses this gap by applying a Siamese Network with EfficientNet-B0 specifically for writer verification, drawing on the preprocessing insights of Kim et al. [12] and the style-awareness perspective of Dehkordi et al. [8].

Table 2.1: Summary of Related Works

Author(s)	Approach	Dataset	Result
Kim et al. (1999)[12]	End-to-end pipeline: line sep., word seg., NN-based recognition, linguistic post-proc.	IAM-style page images	94.5% word seg., 79.8% top-10 recog.
Siddique et al. (2019)[18]	CNN with variable hidden layers	MNIST	99.21%
Ingle et al. (2019) [11]	GRCL/BLSTM + online data pipeline + degradation	IAM Offline	4.0% CER
Balci et al. (2019)[4]	CNN (VGG-19, ResNet) word & char-level + Tesseract seg.	IAM	31% (char-level)
Ahamed et al. (2020) [1]	CNN with data augmentation	Arabic numerals	98.91%
Dehkordi et al. (2003)[8]	36 contour features + PNN style classifier	18-writer dataset	86.5% (legible/illegible)
Hamdan & Sathesh (2021)[10]	Statistical SVM	MNIST, IAM	88.00%
Noubigh et al. (2021)[15]	CNN + BLSTM	KHATT	8% CER
Saqib et al. (2022) [17]	Custom CNN	Kaggle, MNIST	99.64%
Ali & Mallaiah (2022)[ali]	CNN + SVM + Dropout	HACDB	99.85%
Chandio et al. (2022)[5]	CRNN + CTC	Cursive Urdu	95.75%
Coquenot et al. (2023)[7]	FCN Encoder + Transformer	READ 2016	3.43% CER

Chapter 3

Methodology

3.1 Data Preprocessing

The CEDAR signature dataset was preprocessed to ensure consistency and suitability for training the Siamese network model. Initially, all input images were converted to grayscale and inverted so that the signature strokes appear prominently against the background. Each image was then resized while preserving its aspect ratio to fit within a fixed dimension of **224×224 pixels**. To maintain uniformity, zero-padding was applied symmetrically around the image so that all samples have identical dimensions. Finally, each processed image was reshaped to include a single channel.

The dataset consists of both genuine and forged signatures, which were labeled accordingly, where **1 represents genuine** signatures and **0 represents forged** signatures. Additionally, writer IDs were extracted from file names to group signatures belonging to the same individual. The processed data, including images, labels, and writer information, was stored in an **HDF5 (.h5)** file format for efficient storage and faster loading during training. To train the Siamese network, image pairs were generated from the dataset. For each writer, all possible combinations of signature pairs were created, including both genuine–genuine and genuine–forged pairs. Pairs consisting of two forged signatures were excluded to maintain meaningful comparisons. A binary label was assigned to each pair using an XOR operation, where **0 indicates similar (genuine) pairs** and **1 indicates dissimilar (forged) pairs**. The final paired dataset was stored as two input arrays (**S1** and **S2**) along with corresponding labels.

The resulting dataset contained a total of **46,860** signature pairs, with **31,680 dissimilar pairs** and **15,180 similar pairs**. This distribution was further visualized using a count plot to analyze class balance before training.

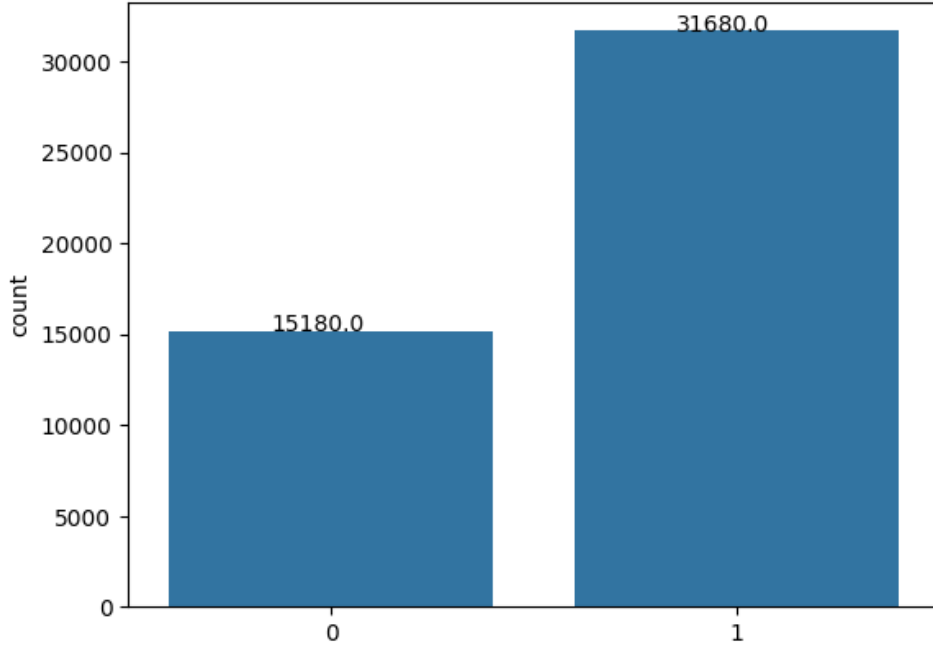


Figure 3.1: Similar V/S Disimilar Count Plot

3.2 Model Architecture

The proposed system is based on a **Siamese Network architecture** designed to learn similarity between two signature images. The architecture consists of two identical subnetworks with shared weights, ensuring that both input images are processed in the same manner to extract comparable feature representations.

Each input signature image of size 224×224 is passed through a feature extraction network based on **EfficientNetB0**, which is pre-trained on the ImageNet dataset. The EfficientNet backbone is used without its top classification layers (*include_top=False*), allowing it to act as a deep feature extractor.

The extracted feature maps are passed through a **Global Average Pooling** layer to reduce spatial dimensions, followed by **Batch Normalization** and a **Dropout layer (rate = 0.5)** to obtain a compact and robust feature embedding.

The same feature extraction network (with shared weights) is applied to both input images, producing two embedding vectors. These embeddings are compared using the **Euclidean distance**, where a smaller distance indicates higher similarity (genuine pair) and a larger distance indicates dissimilarity (forged pair).

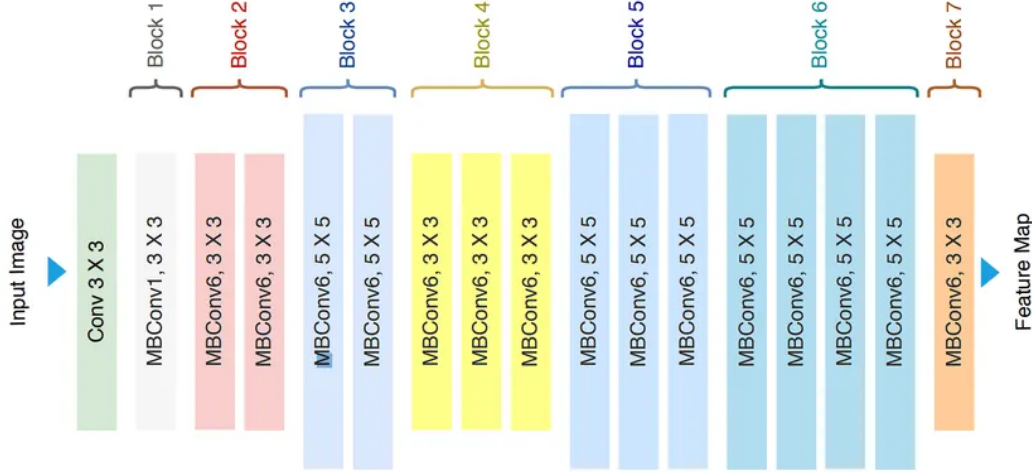


Figure 3.2: EfficientNetB0 Architecture

The computed distance is then passed through a fully connected layer with **sigmoid activation** to produce a probability score indicating whether the pair of signatures is genuine or forged.

Architecture Flow Description

The overall workflow of the proposed model can be summarized as follows:

1. Two input signature images are provided to the network.
2. Both images pass through identical EfficientNetB0-based feature extractors (shared weights).
3. Feature embeddings are generated for each image.
4. The Euclidean distance between the embeddings is computed.
5. The distance is normalized and passed through a dense layer.
6. The final output is a similarity score indicating genuine or forged classification.

The architecture ensures that both inputs are transformed into the same feature space, enabling effective comparison. The use of transfer learning through EfficientNetB0 significantly improves feature extraction capability.

3.3 Training Procedure

After generating the Siamese pairs, the dataset was prepared by addressing class imbalance using a balanced class-weighting strategy. This ensures that

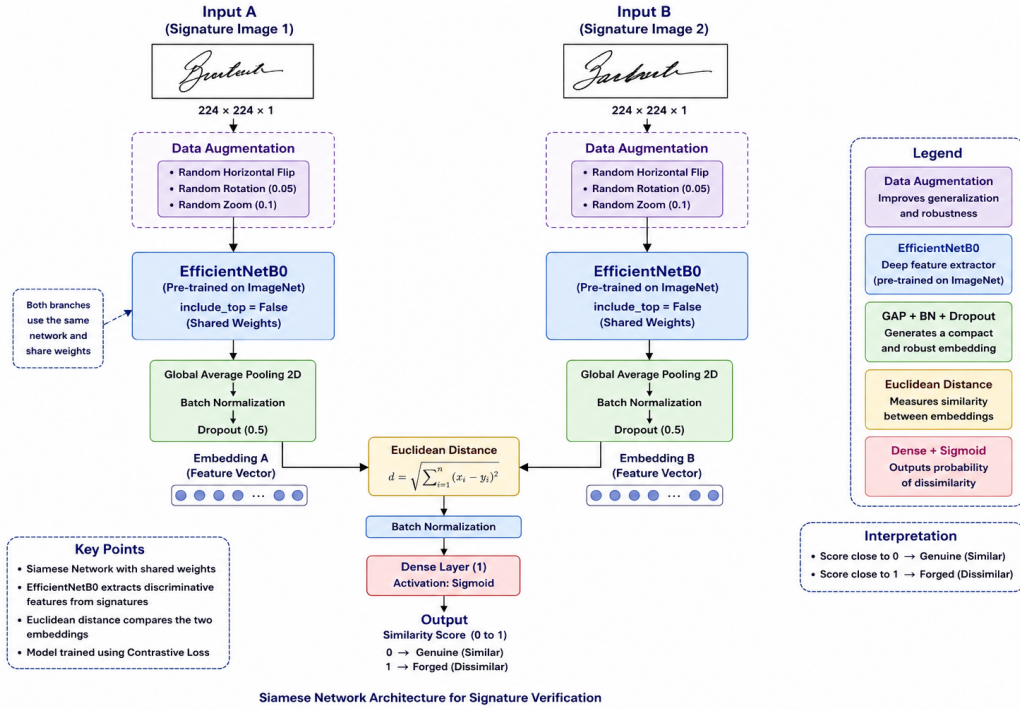


Figure 3.3: Proposed Siamese Network Architecture for Signature Verification. Two input images are processed through identical EfficientNetB0 networks with shared weights. The extracted feature embeddings are compared using Euclidean distance, followed by a classification layer to determine similarity.

both similar and dissimilar pairs contribute equally during training.

The dataset was randomly shuffled and split into **70% training**, **15% validation**, and **15% testing** sets to evaluate model generalization on unseen data. To verify the correctness of the generated pairs, random samples were visualized along with their labels before training.

To improve robustness and generalization, data augmentation techniques such as **random horizontal flipping**, **slight rotation**, and **zooming** were applied. The EfficientNetB0 backbone was initially frozen to retain pre-trained features and later fine-tuned during training.

The model was trained using the **Adam optimizer** with a learning rate of 1×10^{-4} and epsilon value of 1×10^{-8} . The **Contrastive Loss function** was used to learn similarity between signature pairs, with **Binary Accuracy** as the evaluation metric. Training was conducted for **60 epochs** with a batch size of **32** on a **GPU-enabled environment**.

To ensure stable convergence and prevent overfitting, **EarlyStopping** was used to halt training when validation performance stopped improving, **ReduceLROnPlateau** dynamically adjusted the learning rate, and **Mod-**

elCheckpoint saved the best-performing model weights. These techniques improved training stability and overall model performance.

Chapter 4

Result and Discussion

4.1 Environmental Setup

The environmental setup for this project was designed to efficiently support both model training and deployment. For training and experimentation, we utilized the **Kaggle platform**, which provides access to high-performance GPU resources (such as GPU T4). This significantly accelerated the training process of our deep learning model and allowed us to perform multiple experiments with different configurations and data augmentation strategies.

After successfully training the model, we downloaded the trained weights and integrated them into our local development environment. For deployment, we developed a **Streamlit-based web application** on our laptop. All the necessary libraries, including **TensorFlow**, **OpenCV (opencv-python)**, **Matplotlib**, and **Streamlit**, were installed and configured properly.

Additionally, we leveraged our system’s CUDA-enabled GPU to ensure faster inference and smooth execution of the application. This setup enabled a seamless transition from cloud-based training to local deployment, allowing efficient testing and real-time signature verification through the web interface.

4.2 Dataset Description

For training and evaluation of our signature verification model, we used the **CEDAR Signature Dataset**, which is a widely used benchmark dataset for offline signature verification tasks. The dataset consists of signatures collected from 55 different individuals (writers), providing both genuine and forged samples for each writer.

The dataset is organized into two main folders:

- **full_org**: This folder contains a total of 1320 genuine signatures, with 24 genuine samples for each of the 55 writers.

- **full_forg**: This folder contains a total of 1320 forged signatures, with 24 forged samples corresponding to each of the 55 writers.

This balanced distribution of genuine and forged signatures allows the model to effectively learn distinguishing features between authentic and forged signatures. The dataset plays a crucial role in training the Siamese network by providing both positive (genuine-genuine) and negative (genuine-forged) pairs for comparison.

4.3 Evaluation Metrics

To evaluate the performance of our Siamese network for signature verification, we used multiple metrics including **Contrastive Loss**, **Accuracy**, **Euclidean Distance**, and **ROC Curve with AUC**.

4.3.1 Euclidean Distance

The similarity between two signature feature vectors is computed using the Euclidean distance:

$$D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

where x and y represent the feature embeddings of two input signatures. A smaller distance indicates higher similarity (genuine pair), while a larger distance indicates dissimilarity (forged pair).

4.3.2 Contrastive Loss

We used the Contrastive Loss function as proposed by Hadsell et al. (2006) to train the Siamese network:

$$L = (1 - y) \cdot D^2 + y \cdot \max(0, m - D)^2$$

where:

- $y \in \{0, 1\}$ is the label (0 for similar/genuine pair, 1 for dissimilar/forged pair),
- D is the Euclidean distance between embeddings,
- m is the margin (set to 1 in our case).

This loss function minimizes the distance between similar pairs and ensures that dissimilar pairs are separated by at least the margin.

4.3.3 Accuracy, Precision, Recall and F1-Score

The model performance is evaluated using multiple classification metrics including accuracy, precision, recall, and F1-score.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-Score} = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$

where:

- TP = True Positives
- TN = True Negatives
- FP = False Positives
- FN = False Negatives

Accuracy represents the overall correctness of the model, while precision measures how many predicted positive samples are actually correct. Recall indicates how well the model identifies all actual positive samples. The F1-score provides a balance between precision and recall, especially useful in cases of class imbalance.

Accuracy curves for both training and validation sets are plotted to analyze model learning behavior and detect overfitting.

4.3.4 ROC Curve and AUC

To further evaluate classification performance, we used the Receiver Operating Characteristic (ROC) curve, which plots the True Positive Rate (TPR) against the False Positive Rate (FPR):

$$\text{TPR} = \frac{TP}{TP + FN} \quad \text{FPR} = \frac{FP}{FP + TN}$$

The Area Under the Curve (AUC) is computed to summarize the overall performance:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$$

A higher AUC value indicates better model performance. The ROC curve also helps in selecting an optimal threshold for classification.

4.4 Experiments and Results

4.4.1 Experimental Setup

The model was trained using the Siamese network architecture described earlier, with the hyperparameters summarized in Table 4.1. Training was performed on a **GPU-enabled environment** to accelerate computation. The model was optimized using the **Adam optimizer** and trained with the **Contrastive Loss** function, while **Binary Accuracy** was used as the evaluation metric.

Table 4.1: Hyperparameters Used During Training

Hyperparameter	Value
Input Image Size	$224 \times 224 \times 3$
Batch Size	32
Initial Learning Rate	1×10^{-4}
Optimizer	Adam
Maximum Epochs	60
Early Stopping Patience	5
Contrastive Loss Margin m	1.0
Train / Val / Test Split	70% / 15% / 15%

To improve training efficiency and prevent overfitting, **EarlyStopping** was used to halt training when validation performance stopped improving, **ReduceLROnPlateau** dynamically adjusted the learning rate, and **ModelCheckpoint** saved the best-performing model weights. The best model performance was achieved at **epoch 58**, and the corresponding weights were used for evaluation.

4.4.2 Threshold Selection

Since the Siamese network produces a continuous distance score representing the similarity between two signatures, it is necessary to define a threshold to classify whether a pair is **genuine or forged**. To determine the optimal threshold, a range of values between **0.5 and 0.8** was systematically evaluated. For each threshold value, predictions were generated and assessed using the **Receiver Operating Characteristic (ROC) curve**, and the corresponding **Area Under the Curve (AUC)** was computed. The threshold that resulted in the **maximum AUC value** was selected as the optimal decision boundary. This approach ensures an effective balance between the **True Positive Rate (TPR)** and **False Positive Rate (FPR)**, thereby improving the overall classification performance of the model.

4.4.3 Performance Evaluation

The performance of the proposed Siamese network model was evaluated on the test dataset using multiple classification metrics. The model achieved strong performance, demonstrating its effectiveness in distinguishing between genuine and forged signatures.

Metric	Value
Test Accuracy	97.24%
Precision	99.4%
Recall	96.6%
False Acceptance Rate	1.3%
False Rejection Rate	3.4%

Table 4.2: Performance metrics of the proposed model on the test dataset

In addition to these metrics, the model achieved a **False Positive Rate (FPR)** of 0.0130 and a **True Positive Rate (TPR)** of 0.9657, indicating a strong ability to correctly classify both genuine and forged signatures with minimal errors.

4.4.4 Training Analysis

To better understand the learning behavior of the model, training and validation curves for both accuracy and loss were plotted.

The accuracy curve shows a steady increase during training, with both training and validation accuracy converging towards a high value, indicating effective learning. Similarly, the loss curve demonstrates a consistent decrease,

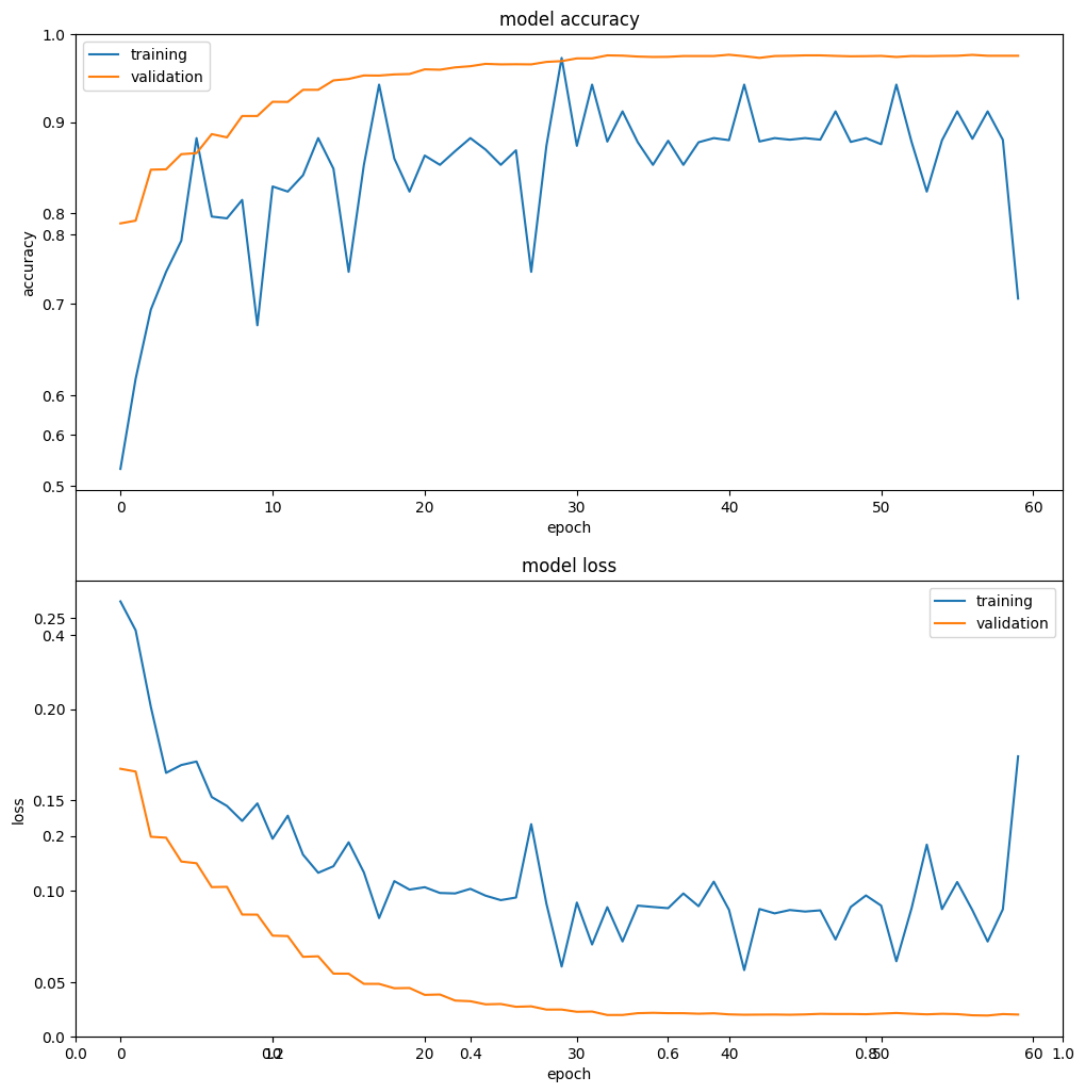


Figure 4.1: Training and validation accuracy and loss curves

suggesting that the model successfully minimizes the contrastive loss over time.

The close alignment between training and validation curves indicates that the model does not suffer significantly from overfitting and generalizes well to unseen data. The use of regularization techniques such as **EarlyStopping** and **ReduceLROnPlateau** further contributed to stable convergence and improved overall performance.

4.4.5 Confusion Matrix

The confusion matrix provides a detailed breakdown of correct and incorrect predictions.

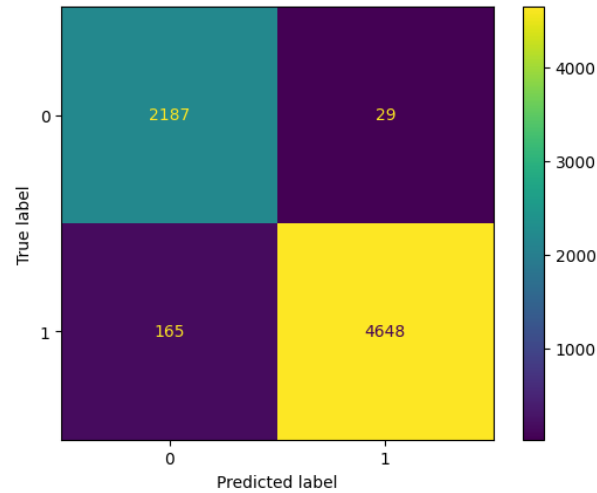


Figure 4.2: Confusion Matrix of the Model

4.4.6 ROC Curve

The ROC curve illustrates the trade-off between True Positive Rate and False Positive Rate.

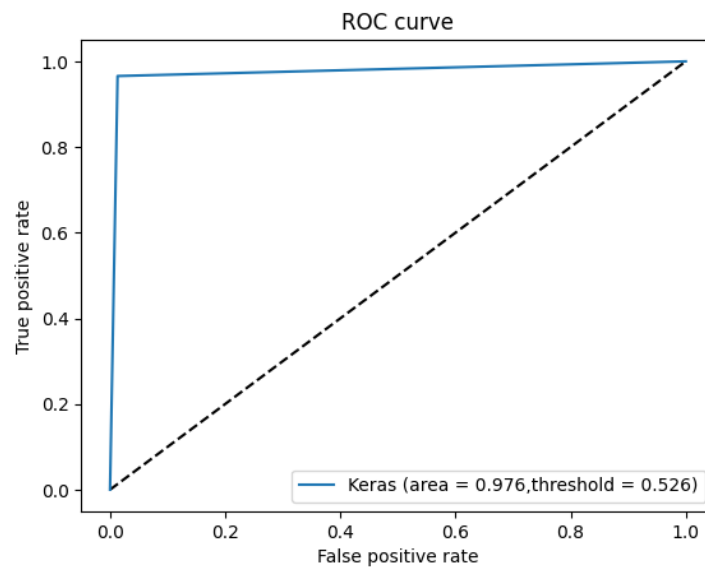


Figure 4.3: ROC Curve showing model performance

4.4.7 Prediction Results

To demonstrate the effectiveness of the model, sample predictions are shown below:



Figure 4.4: Sample Predictions (Genuine vs Forged)

4.4.8 Discussion

The experimental results demonstrate that the proposed Siamese network model performs effectively for signature verification. The high accuracy and AUC score indicate strong discriminative capability between genuine and forged signatures, while the low false positive rate ensures reliability in real-world applications. However, several challenges remain. Intra-writer variability makes verification difficult due to natural variations in signatures, and limited dataset diversity restricts generalization across different scripts and styles. Image noise in scanned documents can affect feature extraction, while pair imbalance complicates training. The model is also sensitive to the verification threshold, impacting FAR and FRR, and may suffer from overfitting when trained on limited data. Additionally, computational cost remains a concern. Addressing these challenges is essential to improve robustness and real-world applicability.

Chapter 5

Web Application

After successfully training the Siamese network model, a user-friendly web application was developed using the **Streamlit framework** to enable real-time signature verification. The application provides an intuitive interface for users to upload and verify signatures without requiring technical knowledge. The application follows a **two-step workflow**. In the first step, the user

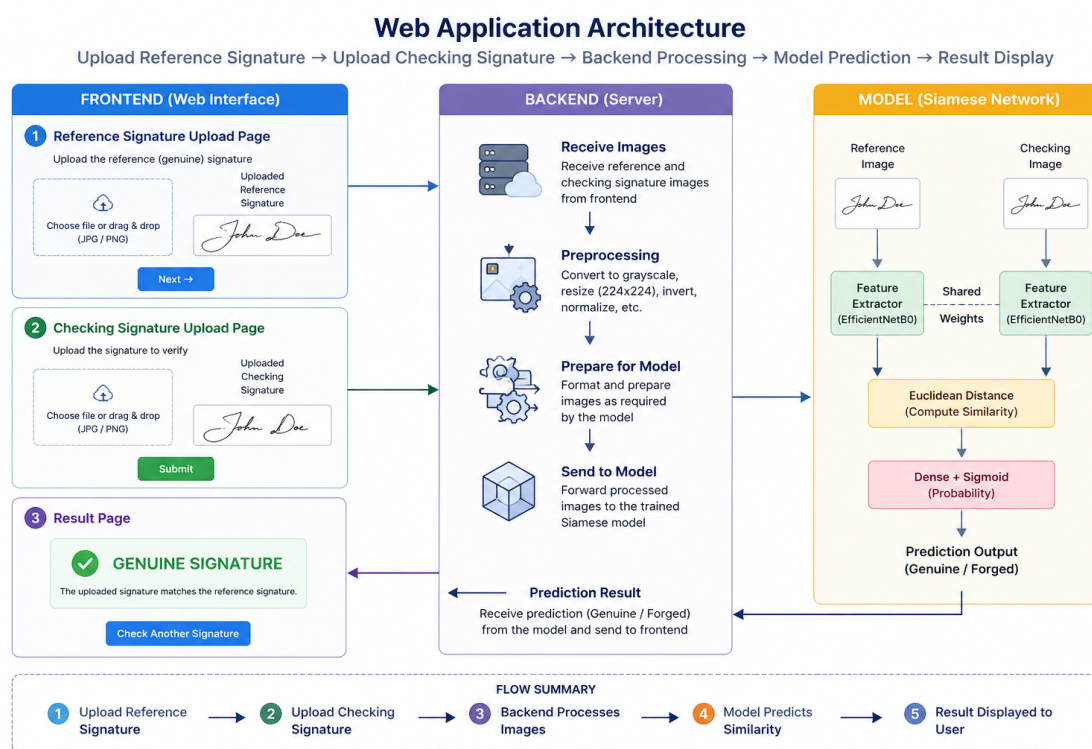


Figure 5.1: Web Application Architecture for Signature Verification System

uploads a **reference signature**, which is assumed to be genuine. This image is displayed on the interface and temporarily stored on the system. Once the reference signature is saved, the application transitions to the verification stage.

In the second step, the user uploads a **test signature** that needs to be verified. Both the reference and test signatures are then processed in the backend using the same preprocessing pipeline as used during model training,

ensuring consistency in input format. The processed images are passed to the trained **Siamese network model**, which computes a similarity (distance) score between the two signatures.

The model output is compared against a predefined **decision threshold** to classify the signature as **genuine or forged**. If the predicted score exceeds the threshold, the signature is classified as forged; otherwise, it is considered genuine. The application also displays a **difference score** along with a confidence level to provide additional insight into the prediction.

To enhance performance, the trained model is loaded using a caching mechanism, ensuring that it is not reloaded repeatedly during user interaction. The application also maintains session state to handle navigation between different steps seamlessly.

User Interface

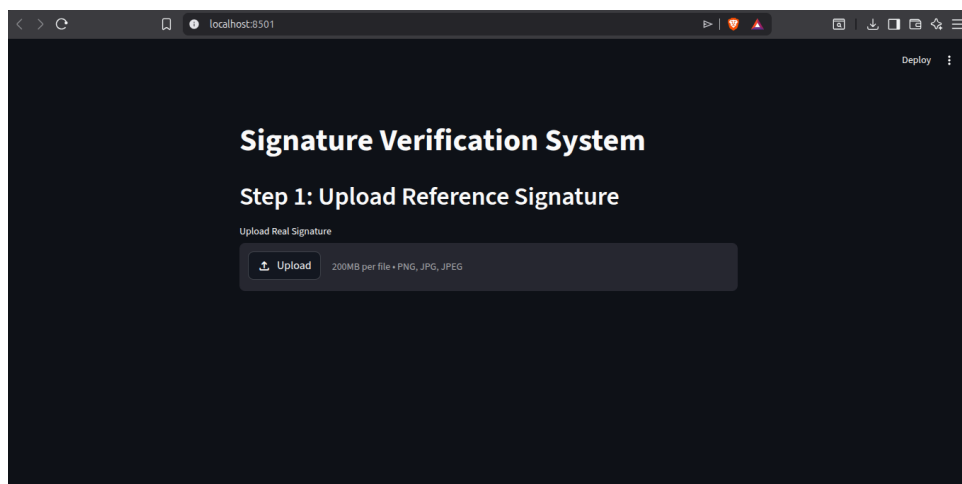


Figure 5.2: Web interface showing signature upload page

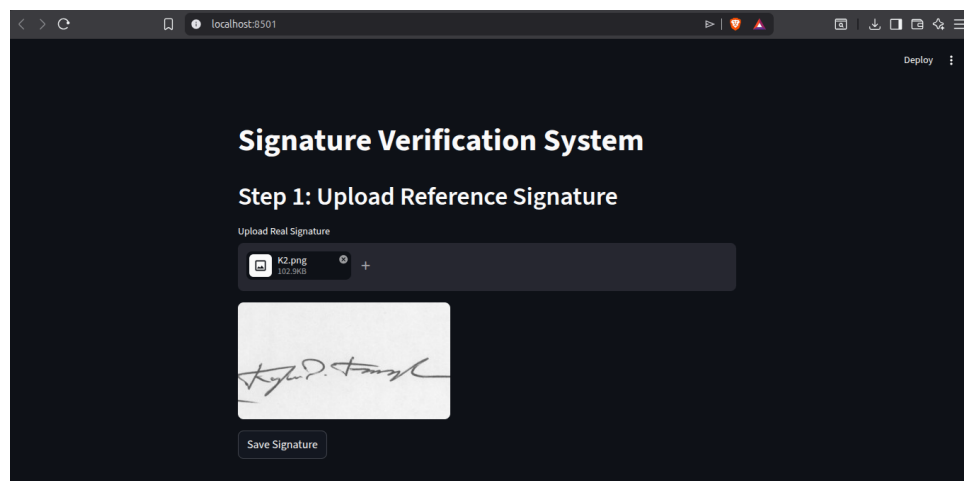


Figure 5.3: Web interface showing signature upload page

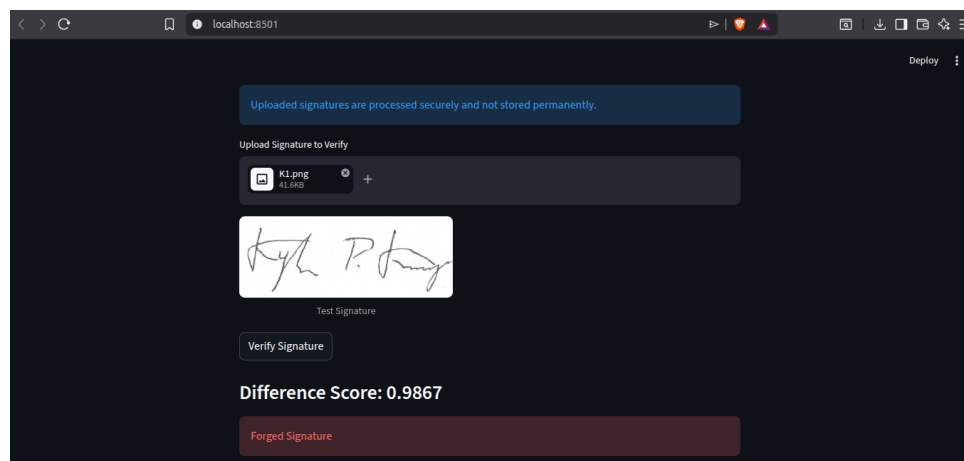


Figure 5.4: Web interface showing signature upload page

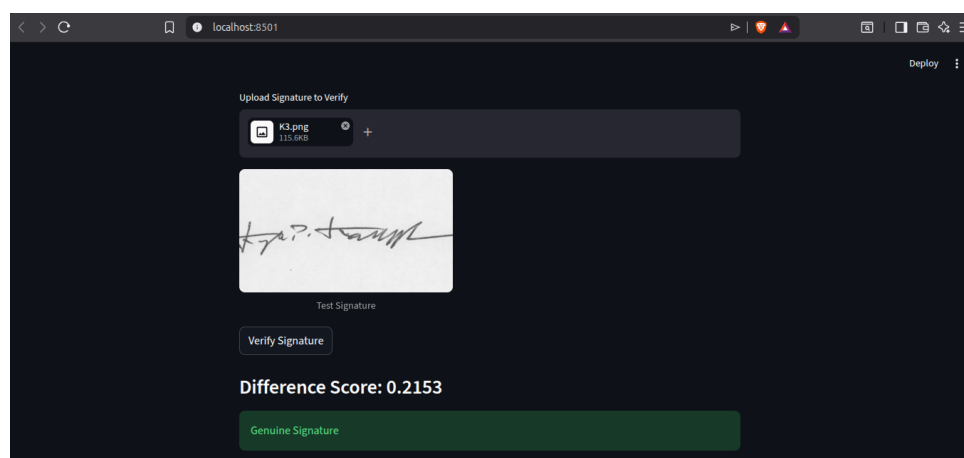


Figure 5.5: Web interface showing signature upload page

Chapter 6

Conclusion

This project successfully developed an offline signature verification system using a Siamese Network architecture with an EfficientNet-B0 backbone. The system effectively learns similarity between signature pairs and demonstrates strong capability in distinguishing between genuine and forged signatures. The experimental results validate the effectiveness of the proposed approach, achieving high accuracy and low error rates, indicating reliable performance for practical applications. The use of contrastive learning combined with transfer learning enables the model to generalize well despite limited dataset size. Additionally, the integration of a Streamlit-based web application demonstrates the feasibility of deploying the system for real-time usage. However, certain challenges remain, including intra-writer variability, dataset limitations, and sensitivity to threshold selection. Addressing these issues will be essential for improving robustness and performance in diverse real-world scenarios. In conclusion, this work highlights the potential of deep learning-based similarity models for biometric verification tasks and provides a strong foundation for future enhancements in signature verification systems.

Chapter 7

Future Work

Future work will focus on addressing key challenges such as intra-writer variability, which makes verification difficult due to natural variations in signatures, and limited dataset diversity that restricts generalization across different scripts and writing styles. Improving robustness against image noise in scanned documents and handling pair imbalance more effectively during training will also be important. Additionally, enhancing **model generalizability** across unseen datasets through **cross-dataset training** and exploring **data fusion** techniques by combining multiple signature datasets can significantly improve performance. Optimizing the verification threshold to balance FAR and FRR, reducing overfitting with better regularization and augmentation strategies, and minimizing computational cost for efficient training and deployment are also essential directions for making the system more robust and scalable in real-world applications.

Bibliography

- [1] P. Ahamed et al. “Handwritten Arabic numerals recognition using convolutional neural network”. In: *Journal of Ambient Intelligence and Humanized Computing* 11 (2020), pp. 5445–5457.
- [2] H. A. Alhamad et al. “Handwritten Recognition Techniques: A Comprehensive Review”. In: *Symmetry* 16 (2024), p. 681.
- [3] A. A. A. Ali and S. Mallaiah. “Intelligent handwritten recognition using hybrid CNN architectures based-SVM classifier with dropout”. In: *Journal of King Saud University – Computer and Information Sciences* 34 (2022), pp. 3294–3300.
- [4] B. Balci, D. Saadati, and D. Shiferaw. *Handwritten Text Recognition using Deep Learning*. 2019.
- [5] A. A. Chandio et al. “Cursive text recognition in natural scene images using deep convolutional recurrent neural network”. In: *IEEE Access* 10 (2022), pp. 10062–10078.
- [6] Y. Chychkarov et al. “Handwritten Digits Recognition Using SVM, KNN, RF and Deep Learning Neural Networks”. In: *Content Management and Interoperability of Services*. Vol. 2864. 2021, pp. 496–509.
- [7] D. Coquenat, C. Chatelain, and T. Paquet. “DAN: A Segmentation-Free Document Attention Network for Handwritten Document Recognition”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (2023), pp. 8227–8243.
- [8] M. E. Dehkordi, N. Sherkat, and T. Allen. “Handwriting style classification”. In: *International Journal on Document Analysis and Recognition (IJDAR)* 6 (2003), pp. 55–74.
- [9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [10] Y. B. Hamdan and A. Sathesh. “Construction of statistical SVM based recognition model for handwritten character recognition”. In: *Journal of Information Technology and Digital World* 3 (2021), pp. 92–107.
- [11] R. R. Ingle et al. “A Scalable Handwritten Text Recognition System”. In: *Proceedings of ICDAR 2019*. IEEE, 2019, pp. 17–24.
- [12] G. Kim, V. Govindaraju, and S. N. Srihari. “An architecture for handwritten text recognition systems”. In: *International Journal on Document Analysis and Recognition (IJDAR)* 2 (1999), pp. 37–44.
- [13] G. Koch, R. Zemel, and R. Salakhutdinov. “Siamese Neural Networks for One-shot Image Recognition”. In: *ICML Deep Learning Workshop*. 2015.
- [14] U. V. Marti and H. Bunke. “The IAM-database: an English sentence database for offline handwriting recognition”. In: *International Journal on Document Analysis and Recognition* 5 (2002), pp. 39–46.
- [15] Z. Noubigh, A. Mezghani, and M. Kherallah. “Contribution on Arabic handwriting recognition using deep neural network”. In: *Hybrid Intelligent Systems*. Springer, 2021, pp. 123–133.

- [16] H. Parikshith et al. “Handwritten character recognition of Kannada language using convolutional neural networks and transfer learning”. In: *IOP Conference Series: Materials Science and Engineering*. Vol. 1110. 2021, p. 012003.
- [17] N. Saqib et al. “Convolutional-neural-network-based handwritten character recognition: an approach with massive multisource data”. In: *Algorithms* 15 (2022), p. 129.
- [18] F. Siddique, S. Sakib, and M. A. B. Siddique. “Recognition of handwritten digit using convolutional neural network in Python with TensorFlow”. In: *Proceedings of ICAEE 2019*. Dhaka, 2019.
- [19] M. Tan and Q. V. Le. “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”. In: *Proceedings of ICML 2019*. 2019.